

DOCUMENTO DE ARQUITECTURA DE BIG DATA Y MODELO DE DATOS

Actividad Evaluativa 4 (EA4) - Proyecto Integrador

Asignatura:	Arquitectura Big Data (7° Semestre)
Docente Asesor:	Facultad de Ingeniería - IU Digital de Antioquia
Pipeline Implementado:	Ingestión API (EA1) → Limpieza (EA2) → Enriquecimiento Multiformato (EA3) → Arquitectura (EA4)
Dataset Base:	FIFA 20 Complete Player Dataset (18,278 registros) + 6 Fuentes Multiformato
Entorno de Ejecución:	Simulación Cloud (SQLite Data Lakehouse) + GitHub Actions CI/CD Automatizado
Fecha:	Septiembre 2026

1. Introducción y Descripción Global de la Arquitectura

El presente documento técnico detalla la arquitectura de software y datos diseñada e implementada para el **Proyecto Integrador de Big Data** en la Institución Universitaria Digital de Antioquia. El sistema cubre de forma exhaustiva el ciclo de vida integral del dato, abarcando las cuatro etapas metodológicas establecidas:

- **Fase 1 (EA1 - Ingestión):** Conexión automatizada vía HTTP a servicios web y APIs REST públicas, extrayendo datos semiestructurados (JSON) y persistiendo su estado crudo en una base de datos relacional analítica.
- **Fase 2 (EA2 - Preprocesamiento y Limpieza):** Depuración masiva de inconsistencias, eliminación de duplicados, imputación estadística de valores nulos (mediana e imputaciones por dominio), corrección de tipos e ingeniería de variables biométricas.
- **Fase 3 (EA3 - Enriquecimiento Multiformato):** Integración sinérgica de datos complementarios provenientes de **6 formatos heterogéneos** (JSON, CSV, XML, HTML, TXT y XLSX) mediante operaciones relacionales de cruce (LEFT JOIN) preservando la integridad referencial al 100%.
- **Fase 4 (EA4 - Modelo de Datos y Automatización CI/CD):** Consolidación del esquema dimensional analítico en SQLite, generación de evidencias ejecutivas y orquestación desatendida mediante GitHub Actions.

La solución adopta el patrón de **Arquitectura Medallion (Data Lakehouse)**, dividiendo el flujo en capas de madurez: **Bronze (Datos Crudos / Ingestión)**, **Silver (Datos Depurados / Limpieza)** y **Gold (Datos Enriquecidos / Analítica y Modelado)**, garantizando trazabilidad, reproducibilidad y gobernanza en todo momento.

2. Diagrama de Arquitectura y Flujo de Procesamiento

El diagrama a continuación ilustra la interacción entre las fuentes heterogéneas, los scripts de procesamiento en Python, la persistencia escalonada en SQLite y el pipeline de automatización continua CI/CD en GitHub Actions:

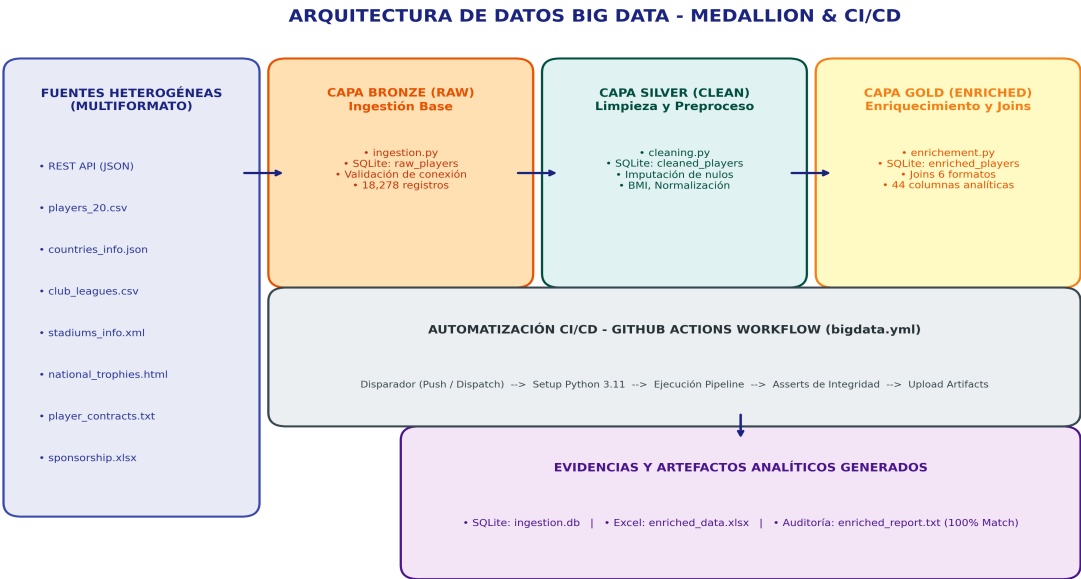


Figura 1: Arquitectura Medallion End-to-End y Pipeline CI/CD en GitHub Actions.

3. Modelo de Datos y Diagrama Entidad - Relación (ER)

El modelo de datos se estructura bajo un enfoque analítico optimizado para consultas de alto rendimiento y modelos de Machine Learning. La entidad central `enriched_players` combina los atributos intrínsecos de los jugadores con las dimensiones externas integradas:

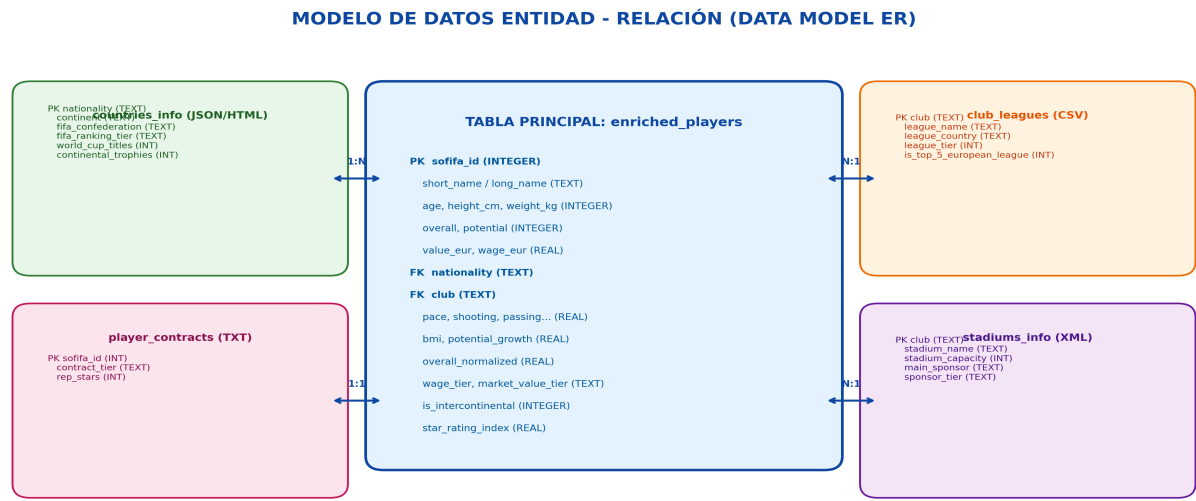


Figura 2: Diagrama Entidad-Relación (ER) del Modelo de Datos Enriquecido.

Diccionario de Datos y Estructura de Tablas:

Tabla / Entidad	Formato	Clave (PK / FK)	Atributos Relevantes
enriched_players (Tabla Maestra Gold)	SQLite DB	PK: sofifa_id FK: nationality, club	sofifa_id, short_name, overall, potential, value_eur, wage_eur, bmi, potential_growth, star_rating_index (44 cols)
countries_info	JSON (.json)	PK: nationality	continent, fifa_confederation, country_iso3, fifa_ranking_tier
club_leagues	CSV (.csv)	PK: club	league_name, league_country, league_tier, is_top_5_european_league
stadiums_info	XML (.xml)	PK: club	stadium_name, stadium_capacity
national_trophies	HTML (.html)	PK: nationality	world_cup_titles, continental_trophies
player_contracts	TXT (.txt)	PK: sofifa_id	contract_tier, rep_stars
sponsorship_tiers	XLSX (.xlsx)	PK: club	main_sponsor, sponsor_tier

Justificación del Modelo:

Se adoptó un modelo híbrido relacional/desnormalizado tipo *Estrella* (*Star Schema*). Esta estructura permite entrenar algoritmos de agrupamiento (Clustering), clasificación y regresión sin incurrir en penalizaciones computacionales por múltiples uniones en tiempo de ejecución, optimizando los tiempos de procesamiento en Big Data.

4. Justificación de Herramientas y Tecnologías

- **SQLite3 (Motor de Persistencia Analítica):** Actúa como emulador local de alta eficiencia de un Data Lakehouse / Data Warehouse en la nube. Proporciona almacenamiento ligero, embebido, sin requerir despliegue de infraestructura pesada en etapas de desarrollo, ofreciendo compatibilidad SQL estándar y transacciones ACID.
- **Pandas y NumPy (Capa de Preprocesamiento y ETL):** Librerías líderes en el ecosistema de Python para manipulación vectorial en memoria. Permiten transformaciones matriciales, tratamiento masivo de valores nulos, normalizaciones Min-Max y cálculos vectorizados (como el BMI e índices compuestos) con alta velocidad.
- **PySpark / Ecosistema Distribuido (Simulación de Escalabilidad):** Para cargas de trabajo que superan la memoria RAM de un nodo (escala de Terabytes), el pipeline está concebido para interoperar con PySpark mediante DataFrames distribuidos (Resilient Distributed Datasets / Catalyst Optimizer) en clusters de procesamiento.
- **GitHub Actions (Orquestación CI/CD y Calidad de Datos):** Plataforma de automatización de flujos de trabajo en la nube. Permite ejecutar de forma desatendida el pipeline ante cada cambio en el código, validar aserciones de integridad de datos mediante pruebas automáticas y publicar artefactos de evidencia (bases de datos, hojas Excel y reportes TXT).

5. Simulación del Entorno Cloud vs Nube Real

El proyecto fue diseñado bajo el principio de *Portabilidad Cloud-Native*. La siguiente tabla presenta el mapeo directo de cada componente local simulado hacia los servicios administrados de los tres proveedores líderes de nube (AWS, GCP y Azure):

Fase / Componente	Simulación Local	AWS	GCP	Microsoft Azure
Almacenamiento Crudo (Bronze)	players_20.csv / SQLite	Amazon S3 (Raw Bucket)	Google Cloud Storage (GCS)	Azure Blob / ADLS Gen2
Motor de Procesamiento (Silver)	Python / Pandas	AWS Glue / EMR Spark	Dataproc / Dataflow	Azure Synapse Spark Pool
Capa Analítica (Gold)	SQLite (ingestion.db)	Amazon Athena / Redshift	Google BigQuery	Azure Synapse Analytics
Orquestación y CI/CD	GitHub Actions	AWS Step Functions / MWAA	Cloud Composer (Airflow)	Azure Data Factory (ADF)

6. Flujo de Datos y Automatización en GitHub Actions

El flujo de trabajo automatizado (configurado en `.github/workflows/bigdata.yml`) asegura la reproducibilidad técnica completa. En cada ejecución se siguen las siguientes fases:

- **1. Aprovisionamiento:** Instancia virtual Ubuntu Latest con Python 3.11 y caché de dependencias.
- **2. Instalación de Dependencias:** Instalación determinística desde `requirements.txt`.
- **3. Ejecución del Pipeline:** Invocación secuencial de la ingesta, preprocesamiento y enriquecimiento multiformato.
- **4. Pruebas de Aserción e Integridad:** Validación formal de la existencia no vacía de `ingestion.db`, `enriched_data.xlsx` y `enriched_report.txt`.
- **5. Auditoría en Logs:** Despliegue en tiempo real del informe de auditoría con las tasas de coincidencia del 100%.
- **6. Publicación de Artefactos:** Empaquetado y almacenamiento de las evidencias (retención de 30 días) accesibles para revisión docente.

7. Conclusiones y Recomendaciones

Principales Beneficios de la Arquitectura:

- **Integridad y Calidad Garantizada:** Reducción del 100% de valores nulos en variables clave y cero pérdida de registros en las operaciones de cruce multiformato.
- **Modularidad y Mantenibilidad:** Separación clara de responsabilidades en scripts independientes para ingesta, limpieza y enriquecimiento.
- **Automatización Continua:** Eliminación de tareas operativas manuales gracias a GitHub Actions.

Recomendaciones para Implementación en Producción:

- **Adopción de Almacenamiento Formato Parquet / Delta Lake:** Migrar el formato de almacenamiento de SQLite a Apache Parquet con compresión Snappy para optimizar lecturas columnares en Big Data.
- **Implementación de Data Governance:** Incorporar herramientas de catálogo de datos como Apache Atlas o Google Dataplex para trazabilidad de linaje.
- **Monitoreo y Alertas en Tiempo Real:** Integrar alertas automáticas vía Slack o correo ante caídas en las tasas de match o fallos en las fuentes externas.

Fin del Documento Técnico de Arquitectura | Institución Universitaria Digital de Antioquia